# Apuntes de Acceso a Datos

UT2. Conexión a bases de datos relacionales (JDBC)

Documento final de estudio: teoría, tablas de repaso y ejemplos prácticos completos.

## Índice


- **Parte 1.** Conceptos y fundamentos de JDBC (tema 2.1)
- **Parte 2.** Sentencias DML / DDL: Statement, PreparedStatement y CallableStatement (tema 2.2)
- **Parte 3.** Procesamiento de resultados: ResultSet y ResultSetMetaData (tema 2.3)
- **Parte 4.** Transacciones y control de errores (tema 2.4)
- **Parte 5.** Ejemplos prácticos completos (preparación, un ejemplo por tema)
- **Parte 6.** Chuleta final y errores frecuentes


## Cómo usar estos apuntes

- Primero leer cada parte, después repasar solo las tablas de **Resumen** al final de cada una.
- Los ejemplos de la Parte 5 forman un mismo proyecto Maven: comparten la clase `ConexionBD` y las tablas del script SQL.
- Los ejemplos están pensados para MySQL; con PostgreSQL o SQLite solo cambian la URL, el driver y algunos tipos del script.

# Parte 1. Conceptos y fundamentos de JDBC

## 1.1 Qué es JDBC

**JDBC (Java Database Connectivity)** es una API estándar de Java que conecta una aplicación con una base de datos relacional (MySQL, PostgreSQL, SQLite...). El desarrollador escribe código Java con interfaces JDBC y el **driver** traduce esas llamadas a comandos del SGBD. Así la lógica de acceso a datos es portable, siempre que exista un driver JDBC para el gestor. Sin JDBC, cada base de datos exigiría su propio lenguaje o protocolo.

### Arquitectura en cuatro capas

| Capa | Descripción | Ejemplo |
|---|---|---|
| Aplicación Java | El programa que realiza las operaciones | Clase `Main` o `App` |
| API JDBC | Interfaces de `java.sql` y `javax.sql` | `Connection`, `Statement`, `ResultSet` |
| Driver JDBC | Traduce las órdenes JDBC al lenguaje del SGBD | `com.mysql.cj.jdbc.Driver` |
| SGBD | La base de datos propiamente dicha | MySQL, PostgreSQL, SQLite |

En el esquema, la aplicación usa la JDBC API y el JDBC Driver Manager (que gestiona los distintos drivers); el driver concreto actúa de puente con la base de datos.

## 1.2 Gestores embebidos e independientes

| | Embebidos | Independientes |
|---|---|---|
| Ejecución | Dentro de la propia aplicación, sin servidor | Servicio/servidor al que se accede por red (localhost o IP) |
| Datos | Un único archivo (`.db`), sin instalación | Gestionados por el servidor |
| Uso | Aplicaciones pequeñas o portátiles | Cliente–servidor, aplicaciones web; concurrencia y escalabilidad |
| Ejemplos | SQLite | MySQL, PostgreSQL |

En ambos casos la conexión desde Java se hace con drivers **tipo 4**; cambian la URL y el driver concreto.

## 1.3 Tipos de drivers

| Tipo | Nombre | Traducción | Externos | Ventajas | Inconvenientes | Estado |
|---|---|---|---|---|---|---|
| 1 | JDBC–ODBC Bridge | JDBC → ODBC → SGBD | Driver ODBC del sistema | Simple | Obsoleto, requiere ODBC | Obsoleto (eliminado desde Java 8) |
| 2 | Native API | JDBC → librería nativa → SGBD | Librerías nativas | Rápido | Depende del SO | Obsoleto, muy raro |
| 3 | Network Protocol | JDBC → middleware → SGBD | Servidor intermedio | Independiente del cliente | Necesita servidor intermedio | En desuso |
| 4 | Pure Java | JDBC → SGBD (directo) | Ninguno | Multiplataforma, rápido, recomendado | Ninguno relevante | En uso (estándar) |

El tipo 4 está implementado 100% en Java y traduce las llamadas JDBC directamente a comandos nativos del gestor. Los ejemplos de tipos 1, 2 y 3 del temario son teóricos.

| SGBD | Driver | URL | Dependencia Maven | Servidor |
|---|---|---|---|---|
| MySQL | `com.mysql.cj.jdbc.Driver` | `jdbc:mysql://localhost:3306/empresa` | `com.mysql:mysql-connector-j` | Sí |

| SGBD | Driver | URL | Dependencia Maven | Servidor |
|---|---|---|---|---|
| PostgreSQL | `org.postgresql.Driver` | `jdbc:postgresql://localhost:5432/empresa` | `org.postgresql:postgresql:42.7.3` | Sí |
| SQLite | `org.sqlite.JDBC` | `jdbc:sqlite:C:/basedatos/empresa.db` | `org.xerial:sqlite-jdbc:3.46.1.0` | No (sin user ni pass) |

En proyectos Maven no hace falta añadir el `.jar` a mano: el conector se descarga del repositorio central al compilar.

**Ojo:** el PDF del temario usa el conector MySQL `8.0.0` en el ejemplo y `9.0.0` en la tabla comparativa. Conviene usar una versión actual.

## 1.4 Tecnologías de persistencia sobre JDBC

Se construyen encima de JDBC (usan internamente un driver tipo 4) y **no lo sustituyen**.

| Tecnología | Descripción | Abstracción |
|---|---|---|
| JDBC | API estándar para conectar, consultar y manipular BD relacionales | Bajo |
| JPA / Hibernate | Mapeo objeto-relacional (ORM). Es la **siguiente unidad de trabajo** | Medio |
| Spring Data JPA | Extiende JPA y automatiza el CRUD. **No se incluye en el currículo** | Alto |
| MyBatis | Mapea consultas SQL personalizadas a objetos Java | Medio |

## 1.5 Flujo típico de una operación JDBC

| Paso | Qué se hace | Código clave |
|---|---|---|
| 1 | Registrar el driver (implícito con tipo 4 y Maven) | `Class.forName("com.mysql.cj.jdbc.Driver")` |
| 2 | Obtener la conexión | `DriverManager.getConnection(url, user, pass)` |
| 3 | Crear la sentencia | `Statement` o `PreparedStatement` |
| 4 | Ejecutar | `executeUpdate()` (INSERT, UPDATE, DELETE, CREATE) o `executeQuery()` (SELECT) |
| 5 | Procesar resultados | `while (rs.next()) { ... }` |
| 6 | Cerrar recursos | try-with-resources |
| 7 | Transacciones (opcional) | `setAutoCommit(false)`, `commit()`, `rollback()` |

### Buenas prácticas

- Usar **try-with-resources** para cerrar conexiones y liberar recursos.
- **No guardar contraseñas en el código**: usar ficheros de configuración.
- Comprobar que el driver está en el **classpath** del proyecto.

### Resumen de la Parte 1

| Concepto | Clave |
|---|---|
| JDBC | API estándar de Java; el driver traduce a cada SGBD |
| Capas | Aplicación → API JDBC → Driver → SGBD |
| Embebido vs independiente | SQLite (un archivo, sin servidor) vs MySQL/PostgreSQL (servidor por red) |
| Driver actual | Tipo 4, Pure Java; los tipos 1, 2 y 3 son obsoletos o en desuso |
| Conexión | `DriverManager.getConnection(url, user, pass)`; SQLite sin user/pass |
| Frameworks | JPA/Hibernate, Spring Data JPA y MyBatis se apoyan en JDBC |

# Parte 2. Sentencias DML / DDL

## 2.1 Los tres tipos de sentencia

Para ejecutar SQL desde Java se usan objetos que hacen de intermediarios entre la aplicación y el SGBD: envían las órdenes, reciben los resultados y gestionan errores. `PreparedStatement` y `CallableStatement` heredan de `Statement`.

```
java.sql.Statement
 ├── PreparedStatement
 └── CallableStatement
```

| Clase | Para qué | Parámetros | Ejemplo |
|---|---|---|---|
| Statement | SQL estático, sin variables | No | `stmt.execute("CREATE TABLE...")` |
| PreparedStatement | SQL precompilado con variables | Sí (?) | `pst.executeUpdate()` |
| CallableStatement | Procedimientos y funciones almacenadas | IN / OUT / INOUT | `cs.execute()` |

### Flujo general

- Obtener la `Connection`.
- Crear el objeto: `con.createStatement()`, `con.prepareStatement(sql)` o `con.prepareCall("{ call procedimiento(?) }")`.
- Ejecutar: `execute()` (cualquier sentencia), `executeQuery()` (SELECT → `ResultSet`), `executeUpdate()` (INSERT/UPDATE/DELETE → nº de filas).
- Liberar recursos (`ResultSet`, `Statement`, `Connection`), preferiblemente con try-with-resources.

## 2.2 Statement

- Sirve para **DDL** (CREATE, ALTER, DROP), **DML** (INSERT, UPDATE, DELETE) y SELECT sin parámetros dinámicos.
- Una sentencia "sin parámetros" tiene el SQL completamente construido antes de ejecutarse.
- **Cuándo usarlo:** comandos simples o puntuales (crear una tabla, borrar todos los registros) sin variables.
- **Inconvenientes:** poco eficiente si se repite y vulnerable a inyección SQL al concatenar datos del usuario.

```
String sql = "SELECT * FROM EMPLEADOS WHERE DEP_NO = 30";                    // válido: valor fijo
String sql = "SELECT * FROM EMPLEADOS WHERE APELLIDO = '" + nombre + "'";   // PELIGRO: inyección SQL
```

| Método | Uso | Devuelve |
|---|---|---|
| `execute(sql)` | Cualquier instrucción SQL | `true` si devuelve un ResultSet, `false` si no |
| `executeQuery(sql)` | Consultas SELECT | Un `ResultSet` |
| `executeUpdate(sql)` | INSERT, UPDATE o DELETE | Nº de filas afectadas |

## 2.3 PreparedStatement

Extiende `Statement`. Usa `?` como marcador de parámetros, que se rellenan después con los métodos `set...`. El SGBD precompila la sentencia.

- **Seguridad:** separa la estructura de la consulta de los datos, lo que evita la inyección SQL.
- **Rendimiento:** se analiza y optimiza una vez y se reutiliza con distintos valores.
- **Código más simple** en operaciones repetitivas (por ejemplo, muchos INSERT).

| Método | Descripción |
|---|---|
| `setInt(pos, valor)` | Asigna un entero |
| `setString(pos, valor)` | Asigna una cadena |
| `setDouble(pos, valor)` | Asigna un decimal |
| `setDate(pos, valor)` | Asigna una fecha SQL (`java.sql.Date`) |
| `executeQuery()` | Ejecuta SELECT y devuelve `ResultSet` |
| `executeUpdate()` | Ejecuta INSERT, UPDATE o DELETE |
| `clearParameters()` | Limpia los valores antes de reutilizar la consulta |

## 2.4 CallableStatement

Extiende `PreparedStatement`. Ejecuta **procedimientos y funciones almacenadas** (bloques de instrucciones SQL guardados en el servidor y ejecutables por nombre). Aporta reutilización de lógica del lado del servidor, comunicación más eficiente y mejor control de integridad y seguridad.

```
"{CALL nombreProcedimiento(?, ?, ?)}"   // procedimiento
"{? = CALL nombreFuncion(?)}"           // función
```

| Tipo | Significado | En Java |
|---|---|---|
| IN | Java envía un valor al procedimiento | `cs.setInt(1, 10)` |
| OUT | El procedimiento devuelve un valor a Java | `registerOutParameter(1, Types.INTEGER)` y luego `getInt(1)` |
| INOUT | Entra un valor, se modifica y se devuelve | `setDouble(1, 5.5)` **más** `registerOutParameter(1, Types.DOUBLE)` |

### Reglas de posición

- Los índices **empiezan en 1** y siguen el orden de los `?`, que coincide con el orden de la definición del procedimiento.
- Con `ejemplo(IN p_id INT, IN p_nombre VARCHAR(50), OUT p_resultado INT)`: `setInt(1, 123)`, `setString(2, "Ana")`, `registerOutParameter(3, Types.INTEGER)`.
- **En funciones**, la posición **1 es siempre el valor devuelto (RETURN)** y las entradas empiezan en la 2.
- En **INOUT**, el mismo índice sirve para la entrada y la salida.
- `registerOutParameter` se llama **antes** de `execute()`.

**Ojo:** en la tabla del PDF aparecen llamadas como `CALL contarEmpleados(@total)`, que usan variables de sesión de MySQL. Desde JDBC se usan `?` con `registerOutParameter`.

### Resumen de la Parte 2

| Caso | Clave |
|---|---|
| SQL fijo y puntual | `Statement` |
| SQL con datos del usuario o repetitivo | `PreparedStatement` (evita la inyección SQL) |
| Procedimiento almacenado | `CallableStatement` con `{CALL ...}` |
| Función almacenada | `{? = CALL ...}`, el índice 1 es el retorno |
| OUT / INOUT | `registerOutParameter` antes de `execute()` |

# Parte 3. Procesamiento de resultados

## 3.1 El objeto ResultSet

Es una **tabla virtual en memoria**: cada fila es un registro, cada columna un campo, y un **cursor interno** recorre las filas una a una. Se obtiene con `executeQuery()`. Se accede a las columnas por nombre o por posición.

| Método | Descripción |
|---|---|
| `next()` | Cursor a la siguiente fila; `false` si no hay más registros |
| `getInt("col")`, `getString("col")`, `getDouble("col")`... | Valor de la columna indicada |
| `beforeFirst()` / `afterLast()` | Cursor antes de la primera / después de la última fila |
| `first()` / `last()` | Primera o última fila (si es desplazable) |
| `absolute(int fila)` | Va a la fila indicada (si es desplazable) |
| `getRow()` | Número de la fila actual |
| `close()` | Cierra el ResultSet y libera recursos |

### Tipos de ResultSet

Por defecto son **solo hacia adelante y de solo lectura**. Se cambia al crear el `Statement`:

```
Statement stmt = con.createStatement(
    ResultSet.TYPE_SCROLL_INSENSITIVE,
    ResultSet.CONCUR_UPDATABLE);
```

| Desplazamiento | Descripción |
|---|---|
| `TYPE_FORWARD_ONLY` | Solo avanzar (por defecto) |
| `TYPE_SCROLL_INSENSITIVE` | Cualquier dirección; **no** refleja cambios hechos en la BD mientras está abierto |
| `TYPE_SCROLL_SENSITIVE` | Cualquier dirección y **refleja** cambios en tiempo real |

| Concurrencia | Descripción |
|---|---|
| `CONCUR_READ_ONLY` | No permite modificar datos (por defecto) |
| `CONCUR_UPDATABLE` | Permite actualizar o eliminar registros desde el propio ResultSet |

## 3.2 ResultSetMetaData

Necesario con consultas dinámicas (generadas por el usuario o desconocidas en compilación): permite descubrir la estructura del resultado en ejecución. Contiene los **metadatos** (datos sobre los datos).

```
ResultSetMetaData meta = rs.getMetaData();
```

| Método | Descripción |
|---|---|
| `getColumnCount()` | Número total de columnas |
| `getColumnName(int i)` | Nombre de la columna i |
| `getColumnTypeName(int i)` | Tipo SQL (VARCHAR, INT, DOUBLE...) |
| `getColumnDisplaySize(int i)` | Tamaño máximo de la columna |
| `isNullable(int i)` | Si admite NULL |
| `isAutoIncrement(int i)` | Si es autoincremental |
| `getTableName(int i)` | Tabla a la que pertenece la columna |

## 3.3 Actualizar, insertar y eliminar desde ResultSet

**Requisitos de un ResultSet actualizable:** (1) la consulta se refiere a **una sola tabla** (sin JOIN, GROUP BY ni agregados); (2) incluye la **clave primaria**; (3) se crea con `TYPE_SCROLL_INSENSITIVE` y `CONCUR_UPDATABLE`.

| Método | Descripción |
|---|---|
| `updateXxx(columna, valor)` | Asigna un nuevo valor al campo |
| `updateRow()` | Guarda en la BD la fila actual |
| `moveToInsertRow()` | Cursor a una fila vacía para preparar una inserción |
| `insertRow()` | Inserta la nueva fila en la tabla |
| `deleteRow()` | Elimina la fila actual de la tabla |

- Usar `PreparedStatement` en consultas con parámetros.
- Comprobar si el registro existe antes de insertar o eliminar y validar entradas.
- No abusar de los ResultSet actualizables: para operaciones grandes, SQL directo o transacciones.

### Resumen de la Parte 3

| Concepto | Clave |
|---|---|
| ResultSet | Tabla virtual con cursor; `next()` para recorrerla |
| Por defecto | Solo hacia adelante y solo lectura |
| Desplazable | `TYPE_SCROLL_INSENSITIVE` (no ve cambios) o `TYPE_SCROLL_SENSITIVE` (los ve) |
| Actualizable | `CONCUR_UPDATABLE` + una sola tabla + clave primaria |
| Metadatos | `rs.getMetaData()`; estructura de columnas en ejecución |

# Parte 4. Transacciones y control de errores

## 4.1 Concepto y propiedades ACID

Una **transacción** es una unidad lógica de trabajo que agrupa una o más operaciones que se ejecutan de forma **atómica**: o se realizan todas, o ninguna. Ejemplo: una transferencia bancaria (restar 200 € al origen y sumarlos al destino); si una falla, la otra no debe aplicarse.

| Propiedad | Nombre | Descripción |
|---|---|---|
| A | Atomicidad | Todas las operaciones se ejecutan completamente o ninguna |
| C | Consistencia | La BD pasa de un estado válido a otro, respetando restricciones y reglas de integridad |
| I | Aislamiento (Isolation) | Las transacciones concurrentes no interfieren entre sí |
| D | Durabilidad | Lo confirmado (commit) es permanente, incluso ante fallos del sistema |

| Fase | Descripción |
|---|---|
| Inicio | Se ejecutan las operaciones bajo control transaccional |
| Validación | El sistema verifica que no se violen reglas de integridad ni bloqueos |
| Confirmación (COMMIT) | Se consolidan los cambios |
| Deshacer (ROLLBACK) | Si hay error, se anulan todos los cambios |

MySQL y PostgreSQL trabajan por defecto con **autocommit = true** (cada INSERT, UPDATE o DELETE se confirma solo). En SQL puro:

```
SET autocommit = 0;       -- Desactiva el autocommit
START TRANSACTION;        -- Inicia la transacción
UPDATE cuentas SET saldo = saldo - 200 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 200 WHERE id = 2;
COMMIT;                   -- Confirma los cambios   (si algo falla: ROLLBACK;)
```

## 4.2 Transacciones en JDBC

El control se hace con el objeto `Connection`: cada conexión tiene su propio contexto transaccional y `autoCommit` está en `true` por defecto.

- Obtener la conexión.
- `setAutoCommit(false)`.
- Ejecutar las operaciones.
- Si todo va bien: `commit()`.
- Si hay error: `rollback()`.
- Restaurar el autocommit (`setAutoCommit(true)`) y cerrar la conexión.

```
Savepoint sp = con.setSavepoint("NOMBRE");
con.rollback(sp);            // deshace lo hecho después del savepoint
con.releaseSavepoint(sp);    // (opcional) libera la marca
con.rollback();              // deshace toda la transacción
con.commit();                // confirma todo
```

## 4.3 Aislamiento y anomalías de concurrencia

El aislamiento controla cómo se "ven" entre sí las transacciones concurrentes. Se fija en la conexión con `con.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED)`.

| Fenómeno | Qué ocurre |
|---|---|
| Lectura sucia (Dirty Read) | Una transacción lee datos **no confirmados** de otra |
| Lectura no repetible | Una fila leída **cambia** entre dos lecturas de la misma transacción (otra la actualizó y confirmó) |

| Fenómeno | Qué ocurre |
|---|---|
| Lectura fantasma (Phantom Read) | **Aparecen o desaparecen filas** que cumplen una condición (otra transacción insertó/eliminó filas del rango) |
| Escritura sucia (Dirty Write) | Dos transacciones escriben sobre cambios no confirmados de la otra (los motores serios la evitan siempre) |

En la tabla, ✔ significa que el fenómeno **puede ocurrir** y ✘ que se **evita**.

| Nivel | ¿Lectura sucia? | ¿No repetible? | ¿Fantasmas? | Uso típico |
|---|---|---|---|---|
| `READ_UNCOMMITTED` | ✔ | ✔ | ✔ | Pruebas; casi nunca en producción |
| `READ_COMMITTED` | ✘ | ✔ | ✔ | Equilibrio práctico (defecto típico en PostgreSQL) |
| `REPEATABLE_READ` | ✘ | ✘ | ✔ | Mayor estabilidad de lectura (defecto típico en MySQL/InnoDB) |
| `SERIALIZABLE` | ✘ | ✘ | ✘ | Máxima integridad; menor concurrencia |

### Elegir el nivel adecuado

- **READ_COMMITTED:** defecto razonable para la mayoría de CRUD.
- **REPEATABLE_READ:** informes que leen varias veces las mismas filas (p. ej., recalcular totales).
- **SERIALIZABLE:** reglas complejas con dependencias entre lecturas y escrituras de rangos, o cuando se necesitan garantías máximas y se acepta menos paralelismo.
- Transacciones **cortas y acotadas**, con buenos índices en las columnas de filtro para minimizar bloqueos.

## 4.4 Savepoint

Marca dentro de una transacción para deshacer solo una parte con `rollback(savepoint)`. **No** tiene sentido en una transferencia bancaria (todo o nada). **Sí** lo tiene al validar un pedido y luego crear su albarán: si falla el albarán, se conserva el pedido validado.

## 4.5 Mini-plantillas

- **Transacción segura:** siempre que se agrupen varias sentencias DML "todo-o-nada"; restaura el autoCommit al final (crucial con **pools de conexiones**).
- **Actualización atómica:** poner la regla de negocio dentro del UPDATE (`saldo = saldo - ?` y `AND saldo >= ?`) en vez de SELECT + UPDATE, para evitar condiciones de carrera. Si afecta **0 filas**, no hay fondos: `rollback()`.
- **Reintentos:** solo para errores de concurrencia (deadlocks, conflictos de serialización, `SQLTransactionRollbackException`), **no** para errores lógicos (FK, formato). `MAX` bajo (2–3) y espera incremental.
- **Savepoint:** en flujos multi-paso con un paso posterior deseable pero no imprescindible; no en operaciones indivisibles.

```
ps.setBigDecimal(1, importe);   // saldo = saldo - ?
ps.setInt(2, idOrigen);         // id = ?
ps.setBigDecimal(3, importe);   // saldo >= ?
```

### Resumen de la Parte 4

| Concepto | Clave |
|---|---|
| Transacción | Unidad atómica: todo o nada |
| Autocommit | `true` por defecto; `setAutoCommit(false)` para controlarla |

| Concepto | Clave |
|---|---|
| Cerrar la transacción | `commit()` si todo va bien, `rollback()` si hay error |
| Niveles | READ_UNCOMMITTED < READ_COMMITTED < REPEATABLE_READ < SERIALIZABLE |
| Defectos | PostgreSQL: READ_COMMITTED; MySQL/InnoDB: REPEATABLE_READ |
| Savepoint | Rollback parcial; no en operaciones indivisibles |
| Reintentos | Solo errores de concurrencia, máximo 2–3 |

# Parte 5. Ejemplos prácticos completos

Los siete ejemplos forman un mismo proyecto Maven con MySQL. Todos usan la clase `ConexionBD` y las tablas del script SQL, y cada uno aplica lo estudiado en su tema. El código se ha comprobado que compila (Java 21); para ejecutarlo hace falta un servidor MySQL con la base `empresa`.

## 5.1 Preparación del proyecto

**pom.xml** (dependencia del driver tipo 4):

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>9.0.0</version> <!-- o la más reciente disponible -->
</dependency>
```

**src/main/resources/db.properties** (así no hay contraseñas en el código):

```
db.url=jdbc:mysql://localhost:3306/empresa
db.user=root
db.password=Password1234
```

**Script SQL** (tablas del proyecto y rutinas almacenadas para el ejemplo 3):

```
CREATE DATABASE IF NOT EXISTS empresa;
USE empresa;

CREATE TABLE IF NOT EXISTS empleados (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50), salario DECIMAL(10,2), dep_no INT);

CREATE TABLE IF NOT EXISTS cuentas (
    id INT PRIMARY KEY, titular VARCHAR(50), saldo DECIMAL(12,2) NOT NULL);
INSERT INTO cuentas VALUES (1, 'Ana', 1000.00), (2, 'Luis', 500.00);

CREATE TABLE IF NOT EXISTS pedidos (
    id INT AUTO_INCREMENT PRIMARY KEY, estado VARCHAR(20) DEFAULT 'NUEVO');
CREATE TABLE IF NOT EXISTS albaranes (
    id INT AUTO_INCREMENT PRIMARY KEY, pedido_id INT NOT NULL,
    FOREIGN KEY (pedido_id) REFERENCES pedidos(id));
INSERT INTO pedidos (estado) VALUES ('NUEVO');

DELIMITER //
CREATE PROCEDURE aumentarSalario(IN p_dep INT, IN p_porc DOUBLE)
BEGIN
    UPDATE empleados SET salario = salario * (1 + p_porc / 100) WHERE dep_no = p_dep;
END //
CREATE PROCEDURE contarEmpleados(OUT p_total INT)
BEGIN
    SELECT COUNT(*) INTO p_total FROM empleados;
END //
CREATE PROCEDURE doblarNumero(INOUT p_valor INT)
BEGIN
    SET p_valor = p_valor * 2;
END //
CREATE FUNCTION calcularSalarioAnual(p_salario DECIMAL(10,2))
RETURNS DECIMAL(10,2) DETERMINISTIC
BEGIN
    RETURN p_salario * 12;
END //
DELIMITER ;
```

`DELIMITER` es un comando del cliente MySQL (Workbench o consola), no de JDBC. Las rutinas se crean una vez desde el cliente.

**ConexionBD.java · Conexión centralizada leyendo db.properties**

```
import java.io.IOException;
import java.io.InputStream;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
```

```
import java.util.Properties;

public class ConexionBD {
    private static final Properties props = new Properties();

    static {
        try (InputStream in = ConexionBD.class.getClassLoader()
                .getResourceAsStream("db.properties")) {
            if (in == null) {
                throw new IllegalStateException("No se encuentra db.properties");
            }
            props.load(in);
        } catch (IOException e) {
            throw new ExceptionInInitializerError(e);
        }
    }

    public static Connection conectar() throws SQLException {
        return DriverManager.getConnection(
                props.getProperty("db.url"),
                props.getProperty("db.user"),
                props.getProperty("db.password"));
    }
}
```

Cada ejemplo abre la conexión con `ConexionBD.conectar()` dentro de un try-with-resources, así que se cierra sola al terminar el bloque.

## 5.2 Ejemplo 1 · Statement (tema 2.2)

DDL, DML con SQL fijo, SELECT sin parámetros y `execute()` con comprobación de si devuelve un `ResultSet`.

**Ej01_Statement.java**

```
import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class Ej01_Statement {
    public static void main(String[] args) {
        try (Connection con = ConexionBD.conectar();
             Statement stmt = con.createStatement()) {

            // DDL: crear la tabla (SQL fijo, sin variables)
            stmt.execute("CREATE TABLE IF NOT EXISTS empleados ("
                    + "id INT AUTO_INCREMENT PRIMARY KEY, "
                    + "nombre VARCHAR(50), "
                    + "salario DECIMAL(10,2), "
                    + "dep_no INT)");

            // DML: executeUpdate devuelve el numero de filas afectadas
            int ins = stmt.executeUpdate("INSERT INTO empleados (nombre, salario, dep_no) "
                    + "VALUES ('Ana', 1500.00, 30)");
            System.out.println("Filas insertadas: " + ins);

            int upd = stmt.executeUpdate(
                    "UPDATE empleados SET salario = salario + 100 WHERE dep_no = 30");
            System.out.println("Filas actualizadas: " + upd);

            int del = stmt.executeUpdate("DELETE FROM empleados WHERE salario < 500");
            System.out.println("Filas borradas: " + del);

            // SELECT sin parametros: executeQuery devuelve un ResultSet
            try (ResultSet rs = stmt.executeQuery(
                    "SELECT * FROM empleados WHERE dep_no = 30")) {
                while (rs.next()) {
                    System.out.println(rs.getInt("id") + " | " + rs.getString("nombre")
                            + " | " + rs.getDouble("salario"));
                }
            }
```

```
            // execute(): true si la sentencia devuelve un ResultSet
            boolean hayResultSet = stmt.execute("SELECT COUNT(*) FROM empleados");
            if (hayResultSet) {
                try (ResultSet rs = stmt.getResultSet()) {
                    rs.next();
                    System.out.println("Total empleados: " + rs.getInt(1));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

## 5.3 Ejemplo 2 · PreparedStatement (tema 2.2)

Reutilización de una misma sentencia con `clearParameters()`, un intento de inyección SQL tratado como simple dato, y UPDATE/DELETE parametrizados.

**Ej02_PreparedStatement.java**

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class Ej02_PreparedStatement {
    public static void main(String[] args) {
        String insertSql =
                "INSERT INTO empleados (nombre, salario, dep_no) VALUES (?, ?, ?)";

        try (Connection con = ConexionBD.conectar()) {

            // 1) Reutilizar la misma sentencia con distintos valores
            try (PreparedStatement pst = con.prepareStatement(insertSql)) {
                String[] nombres = {"Luis", "Maria", "Carlos"};
                double[] salarios = {1200.50, 1800.75, 1650.00};
                int[] deps = {10, 20, 30};
                for (int i = 0; i < nombres.length; i++) {
                    pst.clearParameters();           // limpiar antes de reutilizar
                    pst.setString(1, nombres[i]);
                    pst.setDouble(2, salarios[i]);
                    pst.setInt(3, deps[i]);
                    System.out.println("Insertadas: " + pst.executeUpdate());
                }
            }

            // 2) SELECT parametrizado: el dato nunca se concatena en el SQL
            String entrada = "Ana' OR '1'='1";   // intento de inyeccion SQL
            try (PreparedStatement pst = con.prepareStatement(
                    "SELECT id, nombre FROM empleados WHERE nombre = ?")) {
                pst.setString(1, entrada);          // se trata como un simple texto
                try (ResultSet rs = pst.executeQuery()) {
                    System.out.println(rs.next() ? "Hay coincidencias"
                                                 : "Sin coincidencias (inyeccion evitada)");
                }
            }

            // 3) UPDATE y DELETE parametrizados
            try (PreparedStatement upd = con.prepareStatement(
                    "UPDATE empleados SET salario = ? WHERE id = ?");
                 PreparedStatement del = con.prepareStatement(
                    "DELETE FROM empleados WHERE dep_no = ?")) {
                upd.setDouble(1, 2000.00);
                upd.setInt(2, 1);
                System.out.println("Actualizadas: " + upd.executeUpdate());

                del.setInt(1, 10);
                System.out.println("Borradas: " + del.executeUpdate());
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
```

```
}
```

## 5.4 Ejemplo 3 · CallableStatement (tema 2.2)

Los cuatro casos del tema: procedimiento con IN, con OUT, con INOUT y función con valor de retorno (posición 1).

**Ej03_CallableStatement.java**

```
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Types;

public class Ej03_CallableStatement {
    public static void main(String[] args) {
        try (Connection con = ConexionBD.conectar()) {

            // 1) Procedimiento con parametros IN: aumentarSalario(dep, porcentaje)
            try (CallableStatement cs = con.prepareCall("{CALL aumentarSalario(?, ?)}")) {
                cs.setInt(1, 30);           // IN p_dep
                cs.setDouble(2, 10.0);      // IN p_porc
                cs.execute();
                System.out.println("Salarios del dep. 30 aumentados");
            }

            // 2) Procedimiento con parametro OUT: contarEmpleados(OUT total)
            try (CallableStatement cs = con.prepareCall("{CALL contarEmpleados(?)}")) {
                cs.registerOutParameter(1, Types.INTEGER);   // antes de execute()
                cs.execute();
                System.out.println("Total empleados: " + cs.getInt(1));
            }

            // 3) Procedimiento con INOUT: doblarNumero(INOUT valor)
            try (CallableStatement cs = con.prepareCall("{CALL doblarNumero(?)}")) {
                cs.setInt(1, 5);                            // valor de entrada
                cs.registerOutParameter(1, Types.INTEGER);   // mismo indice para la salida
                cs.execute();
                System.out.println("Resultado: " + cs.getInt(1));   // 10
            }

            // 4) Funcion: la posicion 1 es el valor devuelto (RETURN)
            try (CallableStatement cs = con.prepareCall("{? = CALL calcularSalarioAnual(?)}")) {
                cs.registerOutParameter(1, Types.DECIMAL);   // 1 -> RETURN
                cs.setDouble(2, 1500.00);                    // 2 -> p_salario
                cs.execute();
                System.out.println("Salario anual: " + cs.getDouble(1));   // 18000.0
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

## 5.5 Ejemplo 4 · ResultSet y ResultSetMetaData (tema 2.3)

Cursor desplazable, método genérico que imprime cualquier SELECT gracias a los metadatos, y un ResultSet actualizable (update, insert y delete desde el propio ResultSet).

**Ej04_ResultSet.java**

```
import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;
import java.sql.Statement;

public class Ej04_ResultSet {

    // Consulta dinamica: las columnas se descubren con ResultSetMetaData
    static void imprimirConsulta(Connection con, String sql) throws SQLException {
        try (Statement st = con.createStatement();
             ResultSet rs = st.executeQuery(sql)) {
```

```
            ResultSetMetaData meta = rs.getMetaData();
            int columnas = meta.getColumnCount();
            for (int i = 1; i <= columnas; i++) {          // los indices empiezan en 1
                System.out.print(meta.getColumnName(i)
                        + "(" + meta.getColumnTypeName(i) + ")\t");
            }
            System.out.println();
            while (rs.next()) {
                for (int i = 1; i <= columnas; i++) {
                    System.out.print(rs.getString(i) + "\t");
                }
                System.out.println();
            }
        }
    }

    public static void main(String[] args) {
        try (Connection con = ConexionBD.conectar()) {

            // 1) ResultSet desplazable (no refleja cambios hechos mientras esta abierto)
            try (Statement st = con.createStatement(
                        ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);
                 ResultSet rs = st.executeQuery("SELECT id, nombre FROM empleados")) {
                rs.last();
                System.out.println("Total filas: " + rs.getRow());
                rs.first();
                System.out.println("Primera: " + rs.getString("nombre"));
                rs.absolute(2);
                System.out.println("Fila 2: " + rs.getString("nombre"));
                rs.beforeFirst();
                while (rs.next()) {
                    System.out.println(rs.getRow() + " -> " + rs.getString("nombre"));
                }
            }

            // 2) Metadatos: funciona con cualquier SELECT
            imprimirConsulta(con, "SELECT * FROM empleados");

            // 3) ResultSet actualizable: una sola tabla y con la clave primaria
            try (Statement st = con.createStatement(
                        ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_UPDATABLE);
                 ResultSet rs = st.executeQuery(
                        "SELECT id, nombre, salario, dep_no FROM empleados")) {

                // UPDATE: subir 50 al salario de todos los del dep. 30
                while (rs.next()) {
                    if (rs.getInt("dep_no") == 30) {
                        rs.updateDouble("salario", rs.getDouble("salario") + 50);
                        rs.updateRow();
                    }
                }

                // INSERT: fila nueva (el id es autoincremental)
                rs.moveToInsertRow();
                rs.updateString("nombre", "Marta");
                rs.updateDouble("salario", 1700.00);
                rs.updateInt("dep_no", 20);
                rs.insertRow();
                rs.moveToCurrentRow();

                // DELETE: borrar la fila de Marta
                rs.beforeFirst();
                while (rs.next()) {
                    if ("Marta".equals(rs.getString("nombre"))) {
                        rs.deleteRow();
                    }
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

## 5.6 Ejemplo 5 · Transferencia bancaria (tema 2.4)

Plantilla de transacción segura con actualización atómica (`saldo >= ?`): si el débito afecta 0 filas, se cancela todo con `rollback()`.

**Ej05_Transferencia.java**

```
import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class Ej05_Transferencia {

    public static void transferir(int origen, int destino, BigDecimal importe)
            throws SQLException {
        String debitar =
                "UPDATE cuentas SET saldo = saldo - ? WHERE id = ? AND saldo >= ?";
        String acreditar = "UPDATE cuentas SET saldo = saldo + ? WHERE id = ?";

        try (Connection con = ConexionBD.conectar()) {
            con.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
            boolean autoCommitPrevio = con.getAutoCommit();
            con.setAutoCommit(false);                       // empieza la transaccion
            try (PreparedStatement psD = con.prepareStatement(debitar);
                 PreparedStatement psA = con.prepareStatement(acreditar)) {

                psD.setBigDecimal(1, importe);
                psD.setInt(2, origen);
                psD.setBigDecimal(3, importe);
                if (psD.executeUpdate() == 0) {             // 0 filas: no hay fondos
                    throw new SQLException("Saldo insuficiente o cuenta origen inexistente");
                }

                psA.setBigDecimal(1, importe);
                psA.setInt(2, destino);
                if (psA.executeUpdate() == 0) {
                    throw new SQLException("Cuenta destino inexistente");
                }

                con.commit();                               // todo bien: confirmar
            } catch (SQLException e) {
                try { con.rollback(); } catch (SQLException rb) { e.addSuppressed(rb); }
                throw e;                                    // todo o nada
            } finally {
                try { con.setAutoCommit(autoCommitPrevio); } catch (SQLException ignore) { }
            }
        }
    }

    public static void main(String[] args) {
        try {
            transferir(1, 2, new BigDecimal("200.00"));
            System.out.println("Transferencia realizada");
        } catch (SQLException e) {
            System.out.println("Transferencia cancelada: " + e.getMessage());
        }
    }
}
```

## 5.7 Ejemplo 6 · Savepoint (tema 2.4)

Se valida el pedido (imprescindible) y se intenta crear el albarán (opcional). Si el albarán falla, `rollback(sp)` conserva la validación.

**Ej06_Savepoint.java**

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.sql.Savepoint;

public class Ej06_Savepoint {
```

```
    // Validar el pedido es imprescindible; crear el albaran es deseable pero opcional
    public static void validarPedido(int idPedido) throws SQLException {
        try (Connection con = ConexionBD.conectar()) {
            con.setAutoCommit(false);
            try {
                try (PreparedStatement ps = con.prepareStatement(
                        "UPDATE pedidos SET estado = 'VALIDADO' WHERE id = ?")) {
                    ps.setInt(1, idPedido);
                    ps.executeUpdate();
                }

                Savepoint sp = con.setSavepoint("TRAS_VALIDAR");   // marca

                try (PreparedStatement ps = con.prepareStatement(
                        "INSERT INTO albaranes (pedido_id) VALUES (?)")) {
                    ps.setInt(1, idPedido);
                    ps.executeUpdate();
                } catch (SQLException e) {
                    con.rollback(sp);        // rollback parcial: se conserva la validacion
                    System.out.println("Albaran no creado; el pedido sigue validado");
                }

                con.commit();                // confirma lo conservado
            } catch (SQLException e) {
                con.rollback();              // fallo grave: se deshace todo
                throw e;
            }
        }
    }

    public static void main(String[] args) throws SQLException {
        validarPedido(1);
    }
}
```

## 5.8 Ejemplo 7 · Reintentos ante errores de concurrencia (tema 2.4)

**Ej07_Reintentos.java**

```
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.SQLTransactionRollbackException;

public class Ej07_Reintentos {
    public static void main(String[] args) throws SQLException {
        int intentos = 0;
        final int MAX = 3;                    // mantener bajo (2-3)
        while (true) {
            try (Connection con = ConexionBD.conectar()) {
                con.setAutoCommit(false);
                // ... trabajo transaccional con PreparedStatement ...
                con.commit();
                break;                        // exito: salir del bucle
            } catch (SQLTransactionRollbackException e) {   // deadlock / serializacion
                if (++intentos >= MAX) {
                    throw e;                  // se agotan los intentos
                }
                try {
                    Thread.sleep(200L * intentos);   // espera incremental
                } catch (InterruptedException ignored) { }
            }
        }
    }
}
```

# Parte 6. Chuleta final y errores frecuentes

## 6.1 Qué usar en cada caso

| Necesito... | Uso |
|---|---|
| Crear/alterar tablas o SQL fijo puntual | `Statement` + `execute()` / `executeUpdate()` |
| Insertar, actualizar o borrar con datos del usuario | `PreparedStatement` + `executeUpdate()` |
| Leer datos | `executeQuery()` → `ResultSet` + `while (rs.next())` |
| Llamar a un procedimiento almacenado | `CallableStatement` con `{CALL p(?, ?)}` |
| Llamar a una función almacenada | `{? = CALL f(?)}`; el índice 1 es el retorno |
| Recorrer un resultado hacia atrás o saltar filas | `TYPE_SCROLL_INSENSITIVE` |
| Modificar filas desde el ResultSet | `CONCUR_UPDATABLE`, una tabla y clave primaria |
| Conocer las columnas de una consulta dinámica | `rs.getMetaData()` |
| Varias operaciones todo-o-nada | `setAutoCommit(false)` + `commit()` / `rollback()` |
| Deshacer solo una parte | `Savepoint` + `rollback(sp)` |
| Evitar lecturas de datos no confirmados | `READ_COMMITTED` (o superior) |
| Fallo transitorio (deadlock) | Reintento limitado (2–3) con espera incremental |

## 6.2 Errores frecuentes

- Concatenar datos del usuario en el SQL de un `Statement` → **inyección SQL**. Usar `PreparedStatement`.
- Olvidar `registerOutParameter` antes de `execute()` en parámetros OUT/INOUT.
- En una función, empezar los parámetros de entrada en 1: **el 1 es el RETURN**, las entradas empiezan en 2.
- Contar las posiciones desde 0: en JDBC (`?`, columnas del ResultSet y de los metadatos) **se cuenta desde 1**.
- Dejar el `autoCommit` en `false` al devolver la conexión (sobre todo con pools): restaurarlo en el `finally`.
- Hacer `commit()` sin `rollback()` en el `catch`: el fallo deja la transacción a medias.
- Usar `SELECT` + `UPDATE` para comprobar saldos: provoca condiciones de carrera; mejor `AND saldo >= ?` en el UPDATE.
- Pedir un ResultSet actualizable con JOIN, GROUP BY o sin la clave primaria.
- Reintentar errores lógicos (FK, formato): solo se reintentan los de concurrencia.
- No cerrar `ResultSet`, `Statement` y `Connection`: usar siempre try-with-resources.
- Guardar usuario y contraseña en el código en lugar de en un fichero de configuración.

## 6.3 Las cinco ideas que hay que llevarse

- JDBC = API estándar + **driver tipo 4** que traduce al SGBD.
- Tres sentencias: `Statement` (fijo), `PreparedStatement` (parámetros, seguro), `CallableStatement` (rutinas almacenadas).
- `ResultSet` es una tabla en memoria con cursor; por defecto, solo hacia delante y solo lectura.
- Una transacción es **todo o nada**: `setAutoCommit(false)` → `commit()` / `rollback()`.
- Todo recurso se cierra con **try-with-resources**.